

Lab Sheet: Designing and Implementing the EC-1 Microprocessor

An 8-bit General-Purpose Microprocessor in Logisim-Evolution

Computer Organization / Digital Systems Laboratory

1 Overview

In Lecture 10 you saw the **EC-1**, a minimal 8-bit general-purpose microprocessor: a Program Counter, an Instruction Register, an Accumulator, a small ROM, and a hardwired Control Unit that walks the datapath through *fetch*, *decode* and *execute*. The ISA and the datapath were fully specified in lecture – you will build those directly in Logisim-Evolution. The **Control Unit**, however, you will **design yourself**: given the ISA and the instruction cycle, you will work out what states are needed, draw the state diagram, encode it, and derive the next-state and output equations, before comparing against the reference design and building it.

By the end of the lab your EC-1 will count down from 10 to 1 on its **Output** port, entirely under the control of hardware you designed and built yourself.

Learning Outcomes

By the end of this lab you will be able to:

- Translate a datapath block diagram (registers, incrementer/decrementer, muxes, tri-state buffer) into Logisim components.
- **Design** a hardwired FSM control unit from an ISA and an instruction-cycle description: choose states, draw the state diagram, encode it, and derive Boolean next-state/output equations.
- Explain why a synchronous memory (e.g. an FPGA's Block-RAM IP) can need an extra idle state after a jump, and why Logisim's own (combinational) ROM does not.
- Hand-assemble an EC-1 program, load it into ROM, and verify its execution cycle by cycle.

2 The EC-1 Instruction Set

EC-1 is an 8-bit-data, 4-bit-address machine: 16 ROM locations, each holding one 8-bit instruction word. Only **3 opcode bits** (IR[7:5]) are needed for 5 instructions; the remaining 5 bits (IR[4:0]) are either don't-care, or (for JNZ) a 4-bit jump address in IR[3:0].

Instruction	Mnemonic	Encoding	Operation
Input	IN A	011 xxxxx	$A \leftarrow \text{Input}$
Output	OUT A	100 xxxxx	$\text{Output} \leftarrow A$
Decrement	DEC A	101 xxxxx	$A \leftarrow A - 1$
Jump if Not Zero	JNZ addr	110 xaaaa	if $A \neq 0$: $PC \leftarrow \text{aaaa}$
Halt	HALT	111 xxxxx	stop execution

Opcodes 000, 001, 010 are unused (treat as no-op: return to **FETCH**). **x** = don't-care; **aaaa** = a 4-bit ROM address.

3 EC-1 Datapath

Figure 1 is the datapath diagram from Lecture 10 – build exactly this in Logisim. Every control signal ($IRload$, $PCload$, $JNZmux$, $INmux$, $Aload$, $OutE$, $Reset$, $Clock$) enters from the left; you will design the logic that generates them in §4.

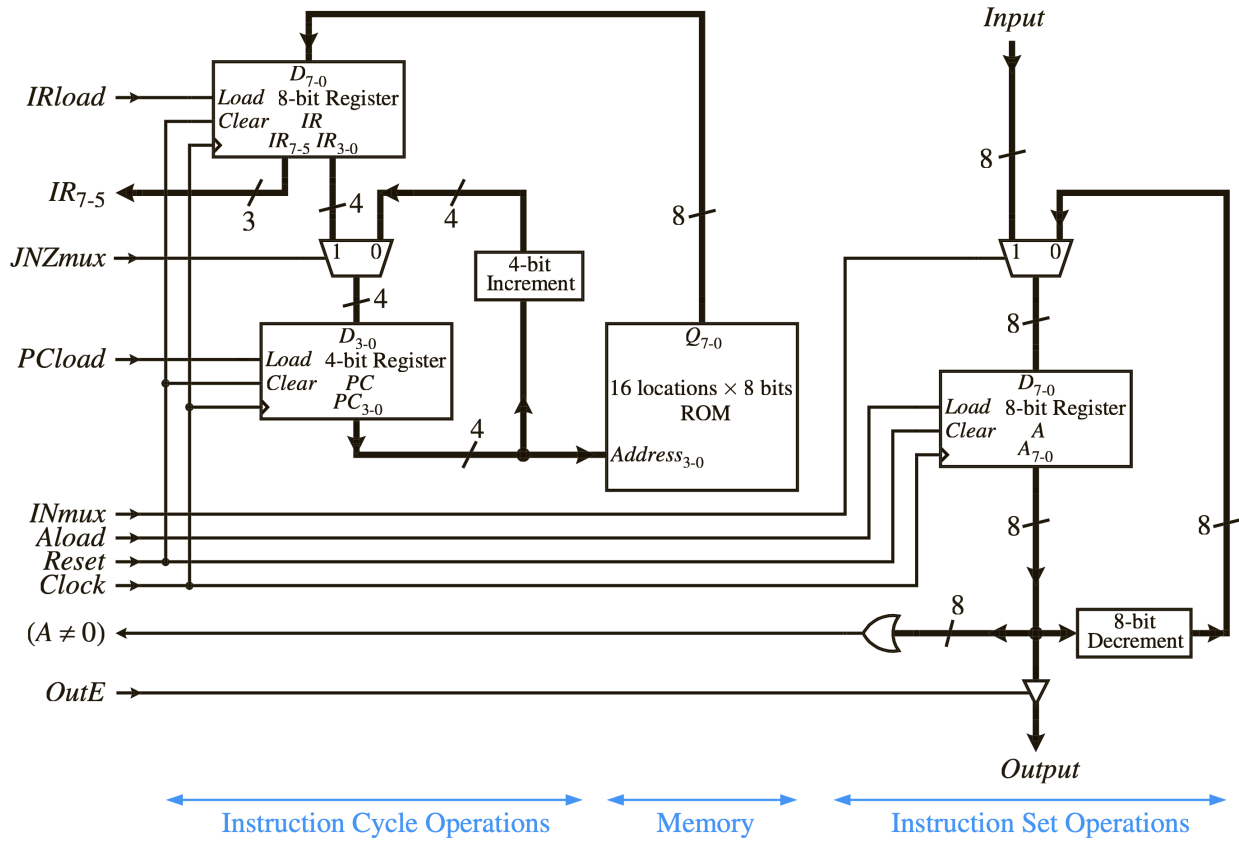


Figure 1: EC-1 datapath (Lecture 10, Figure 10-1). Left: Instruction Cycle Operations (IR , $JNZmux$, PC , the 4-bit Incrementer). Centre: the 16×8 program **ROM**, addressed *directly* by PC – there is no MAR. Right: Instruction Set Operations ($INmux$, the Accumulator A , the 8-bit Decrementer, the zero-check producing $(A \neq 0)$, and the tri-state output buffer gated by $OutE$).

Component Summary

Block	Purpose
PC	4-bit register. D comes from JNZmux: input 0 is $PC + 1$ (from the 4-bit Incrementer), input 1 is IR[3:0] (jump target). Loads when PCload is asserted. Its output is ROM's <i>only</i> address input.
IR	8-bit register, loaded from ROM's data output when IRload is asserted. IR[7:5] (opcode) goes to the Control Unit; IR[3:0] goes to JNZmux input 1.
ROM	16×8 -bit program memory, addressed directly by PC.
A (Accumulator)	8-bit register. D comes from INmux: input 1 is the external Input port, input 0 is the 8-bit Decrementer's output ($A - 1$). Loads when Aload is asserted.
Zero check	An 8-input OR (or NOR, inverted) over A, producing ($A \neq 0$) for the Control Unit's JNZ test.
Tri-state buffer	Drives A onto the Output port only when OutE is asserted; high-impedance otherwise.

Task 1 — Plan before you build. Before opening Logisim, study Figure 1 and label every wire's width on a printout or sketch. Confirm you can answer: *which signal drives ROM's address input, and does that ever change during DEC, IN or OUT?* (It shouldn't – only JNZ and FETCH touch PC.)

4 Designing the EC-1 Control Unit

The Control Unit is what turns the static datapath of Figure 1 into a machine that actually executes programs. It is a **hardwired FSM**: on every clock edge it moves to a new state and asserts a fixed set of control signals, following the classic *fetch* $\rightarrow$ *decode* $\rightarrow$ *execute* instruction cycle:

1. **Fetch:** read the instruction at ROM[PC] into IR; advance PC.
2. **Decode:** look at IR[7:5] and work out which operation to perform.
3. **Execute:** assert exactly the control signals that operation needs, for exactly one clock cycle, then return to **Fetch**.

You will now design this FSM yourself, from the ISA in §2 and the datapath in §3 – *before* looking at the reference design.

Task 2 — Identify the states. List every state your FSM needs. Start with FETCH and DECODE. Then, for each of the 5 instructions in §2, decide: does executing it need its *own* state (because it asserts control signals that no other instruction asserts, in a way that can't share a cycle with DECODE)? What about the 3 unused opcodes – do they need a state of their own?

Task 3 — Draw your own state diagram. On paper, draw the states from Task 2 as circles and draw an arrow for every transition, labelled with the condition that causes it (an opcode value, or nothing if unconditional). Pay particular attention to JNZ: how many *different* next-states can it lead to, and under what condition? Do **not** look ahead until you've drawn this.

Reference Design

Figure 2 is the state diagram from Lecture 10. Compare it against what you drew.

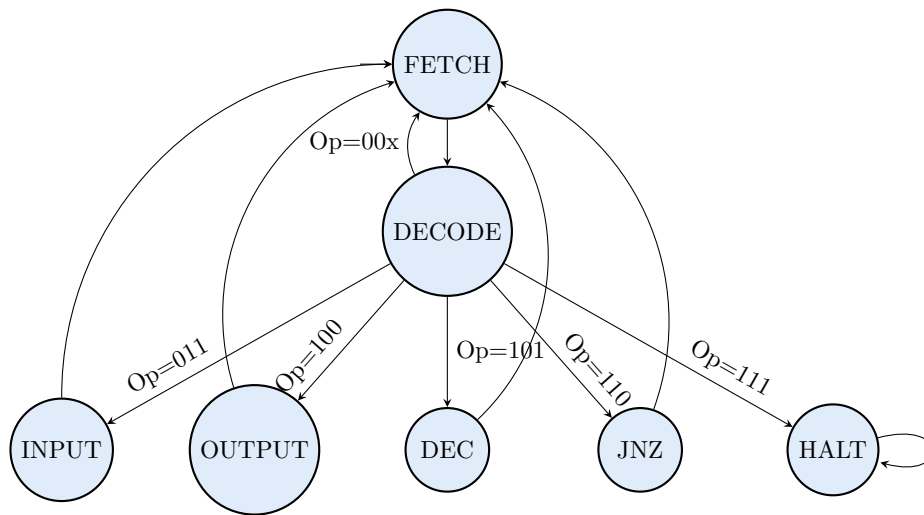


Figure 2: EC-1 Control Unit FSM (Lecture 10, Figure 10-3). Notice JNZ transitions straight back to FETCH, exactly like INPUT, OUTPUT and DEC – there is **no** extra WAIT state here. (§4 explains why lecture’s FPGA/VHDL version needs one and this Logisim version does not.)

Why no Wait state?

The lecture also shows a *second* version of this FSM with an extra WAIT state inserted between JNZ and FETCH. That version targets an FPGA’s **synchronous** Block-RAM IP: the address must be stable *before* the clock edge that captures it, and data appears *after* it. If JNZ updates PC and the FSM moves straight to FETCH on the *same* edge, FETCH would sample memory using the address from *before* that edge – the old PC, not the jump target.

Logisim-Evolution’s built-in ROM component reads combinatorially: it has no clock input at all, and its output tracks its address input immediately (modulo gate delay). Once JNZ loads the new value into PC, ROM’s data output already reflects it well before the next clock edge – there is no hazard to wait out. That is why Figure 2 (the version you will build) omits WAIT entirely.

Task 4 — Discuss. In one paragraph, explain *in your own words* why the FPGA/VHDL version of EC-1 needs a WAIT state and your Logisim version does not. What specifically would break if you deleted WAIT from the VHDL version but kept everything else the same?

Encoding the States

Figure 2 has 7 states; a 3-bit state register $Q_2Q_1Q_0$ can encode up to 8. Lecture uses this encoding:

$Q_2Q_1Q_0$	State	What happens
000	FETCH	$IR \leftarrow ROM[PC]$, $PC \leftarrow PC + 1$.
001	DECODE	Inspect $IR[7:5]$; no datapath signals asserted.
011	INPUT	$A \leftarrow \text{Input}$.
100	OUTPUT	$\text{Output} \leftarrow A$.
101	DEC	$A \leftarrow A - 1$.
110	JNZ	If $A \neq 0$: $PC \leftarrow IR[3:0]$; either way, return to FETCH .
111	HALT	Stop (self-loop).

Code 010 is unused. Notice states 011–111 are exactly the opcode $IR[7:5]$ values 011–111 from §2: **DECODE can jump straight to a state numerically equal to the opcode**. That single design choice is what makes the next-state logic small enough to build by hand.

Task 5 — Derive the next-state equations. Using $\text{Fetch} = \bar{Q}_2\bar{Q}_1\bar{Q}_0$, $\text{Decode} = \bar{Q}_2\bar{Q}_1Q_0$, $\text{Input} = \bar{Q}_2Q_1Q_0$, $\text{Output} = Q_2\bar{Q}_1\bar{Q}_0$, $\text{Dec} = Q_2\bar{Q}_1Q_0$, $\text{JNZ} = Q_2Q_1\bar{Q}_0$, $\text{Halt} = Q_2Q_1Q_0$, and $O_2O_1O_0 = \text{IR}[7:5]$, derive Boolean expressions for D_2, D_1, D_0 (the state register's next value) *yourself* before checking the box below. Work state-by-state: what should $D_2D_1D_0$ equal when the machine is currently in **FETCH**? In **DECODE** with $O_2O_1O_0=101$? In **JNZ**? In **INPUT**? (Hint: for every state *other than* **FETCH**, **DECODE** and **HALT**, the answer is simply $D_2D_1D_0 = 000$ – make sure your equations produce that without you having to add an explicit term for it.)

Check your answer:

$$D_2 = \text{Decode} \cdot O_2 + \text{Halt}$$

$$D_1 = \text{Decode} \cdot O_1 \cdot (O_2 + O_0) + \text{Halt}$$

$$D_0 = \text{Fetch} + \text{Decode} \cdot O_0 \cdot (O_2 + O_1) + \text{Halt}$$

Task 6 — Derive the output equations. For each control signal (**IRload**, **PCload**, **INmux**, **Aload**, **JNZmux**, **OutE**, **Halt**), decide in which state(s) it must be 1, and write the corresponding sum-of-state expression. **PCload** needs care: it must be asserted in **FETCH** (to advance PC) *and* in **JNZ**, but only when the jump is taken.

Check your answer:

$$\text{IRload} = \text{Fetch}$$

$$\text{INmux} = \text{Input}$$

$$\text{JNZmux} = \text{JNZ}$$

$$\text{PCload} = \text{Fetch} + \text{JNZ} \cdot (A \neq 0)$$

$$\text{Aload} = \text{Input} + \text{Dec}$$

$$\text{OutE} = \text{Output}$$

$$\text{Halt} = \text{Halt}$$

Task 7 — Verify by hand. Trace state **JNZ** ($Q=110$) through *your* D_2, D_1, D_0 equations with $O_2O_1O_0=110$, once assuming $A \neq 0$ and once assuming $A = 0$. Both cases should give $D_2D_1D_0 = 000$ (**FETCH**) – confirm that **PCload**'s equation, not the next-state logic, is what makes the jump actually happen (or not).

5 Build Tasks

Work through the tasks in order, testing each block in isolation (Poke tool + a manual clock) before wiring it into the whole machine.

Task 8 — ROM. Add a 16×8 -bit **ROM** component (Memory library). Leave it empty for now – you will load the countdown program in Task 12.

Task 9 — PC side. Build the 4-bit PC register, the 4-bit Incrementer (a 4-bit Adder with one input tied to 0001, or Logisim's arithmetic Incrementer if available), and the **JNZmux** (2:1, 4-bit). Wire PC's output directly to ROM's address input – confirm there is no separate address register anywhere in this path.

Task 10 — IR. Build the 8-bit IR register. Split its output into **IR[7:5]** (opcode, 3 bits) and **IR[3:0]** (address, 4 bits) using Logisim's Splitter component.

Task 11 — Accumulator side. Build the 8-bit A register, the 8-bit Decrementer (an 8-bit Adder/Subtractor, or a dedicated Decrementer), the **INmux** (2:1, 8-bit), the Zero-check (an 8-input OR gate; its output *is* $(A \neq 0)$ directly – no inverter needed), and a Tri-state Buffer (Logisim's Buffer/Transistor library) on A's output, enabled by **OutE**.

Task 12 — Control Unit. Build the 3-bit state register (three D flip-flops, or a Logisim Register), the state-decoder gates (Fetch, Decode, ...), and the next-state / output logic directly from *your*

own equations from Tasks 5–6. Wire ($A \neq 0$) in from the Accumulator side and IR[7:5] in from IR.

Task 13 — Assemble and load the program. Wire Tasks 8–12 together following Figure 1. Hand-assemble the countdown program below, then use ROM’s *Load Image* (right-click → “Load Image”) to load it.

```
Addr Hex Assembly
00 60 IN A ; Input to A
01 80 OUT A ; Output A
02 A0 DEC A ; Decrement A
03 C1 JNZ 01 ; Jump to 01 if A != 0
04 E0 HALT ; Halt execution
05-0F 00 (unused)
```

Logisim’s plain-text image format (paste into ROM’s *Edit Contents*, or save as a `.txt` file and *Load Image*):

```
v2.0 raw
60 80 a0 c1 e0 00 00 00 00 00 00 00 00 00 00
```

Task 14 — Run it. Set the Input pin to 10 (decimal; 0A hex) with the Poke tool. Single-step the clock and confirm Output shows 10, 9, 8, ..., 1 in turn, then the Halt signal goes high and the state register stops changing. ☐ Output sequence matches. ☐ Machine halts cleanly (no further FETCH).

Task 15 — Your own program. Modify the ROM image to count down from a different starting value (poke a new Input), or write a program that outputs every value from a starting count down to zero *twice* (hint: you’ll need more ROM locations and possibly a second jump target).

6 Deliverables

- Your own state diagram (Task 3), *before* you saw Figure 2, plus a short note on how it differed (if it did).
- Your derivations for Tasks 5–7 (next-state equations, output equations, hand-verification), and your answer to Task 4 (Wait-state discussion).
- Your completed `.circ` file: datapath and Control Unit in one top-level EC-1 circuit.
- The ROM image / assembly listing for Tasks 13 and 15.

7 Marking Guide

Criterion	Marks
Own state diagram (Task 3) is complete and correct before comparison	10
Next-state and output equations correctly derived (Tasks 5–6) and verified (Task 7)	20
Wait-state discussion (Task 4)	5
PC, Incrementer, JNZmux correctly built and independently testable	10
IR correctly built, opcode/address split correct	10
ROM addressed directly by PC (no MAR), image loads correctly	5
Accumulator side (A, Decrementer, INmux, Zero-check, tri-state) correct	15
Control Unit correctly implements your equations in hardware	15
Countdown program runs correctly end-to-end, own program (Task 15)	10
Total	100

Extension (Optional, not marked)

The lecture's **EC-2** extends this design to a full Von Neumann machine: 8 instructions, a 32×8 unified **RAM** (code and data together), a real **ALU** (add/subtract), and a **Meminst** mux that – just like **JNZmux** here – chooses between **PC** and **IR**'s address field, but this time to address *data*, not just to fetch instructions. Once EC-1 is working, consider sketching what changes, including whether EC-2's synchronous-vs-asynchronous memory question comes back into play.